Object-Oriented Programming
Parametric Polymorphism

Filip Křikava

https://courses.fit.cvut.cz/BI-OOP/

Any questions?

Warmup...

Can I force the call to `this.inc()` in `inc2` method to be always the one from the `Counter` class?

```
class Counter:
  var x: Int = 1

  def inc(): Unit =
    x += 1

  def inc2(): Unit =
    this.inc()
    this.inc()
```

```
class SpecialCounter extends Counter:
  override def inc(): Unit =
    x += 10

val c: Counter = SpecialCounter()
c.inc2()
println(c.x) // I want 3 here
```

Scala / Java: Three Options

```
class SpecialCounter extends Counter:
  override def inc(): Unit = x += 10
```

1. final

```
class Counter:
  var x = 1
  final def inc(): Unit =
    x += 1

  def inc2(): Unit =
    inc()
    inc()
```

No subclass override.
The `SpecialCounter` no longer compiles

2. private

```
class Counter:
  var x = 1
  private def inc(): Unit =
    x += 1

  def inc2(): Unit =
    inc()
    inc()
```

inc is not public.
The `SpecialCounter` no longer compiles

3. private helper

```
class Counter:
  var x = 1
  private def baseInc(): Unit =
    x += 1

  def inc(): Unit = baseInc()

  def inc2(): Unit =
    baseInc()
    baseInc()
```

inc stays overridable.

Java has the same choices. No ordinary source syntax selects Counter.inc() if it remains overridable.

C++: Qualified Base-Class Call

```
struct Counter {
  int x = 1;
  virtual void inc() { ++x; }

  void inc2() {
    Counter::inc();
    Counter::inc();
  }
};

struct SpecialCounter : Counter {
  void inc() override { x += 10; }
};

SpecialCounter c;
c.inc2(); // x == 3
```

Counter::inc()

Explicit class qualification suppresses virtual dispatch.

`this->inc()` would dispatch dynamically.

`inc()` can still be overridden.

Python: Explicit Class Lookup

```
class Counter:
    def __init__(self):
        self.x = 1

    def inc(self):
        self.x += 1

    def inc2(self):
        Counter.inc(self)
        Counter.inc(self)

class SpecialCounter(Counter):
    def inc(self):
        self.x += 10

c = SpecialCounter()
c.inc2()  # x == 3
```

Counter.inc(self)

`self.inc()` uses dynamic method lookup.

`Counter.inc(self)` gets the function from `Counter` and passes self explicitly.

Python classes can be monkey-patched.



https://kahoot.it/?pin=245111

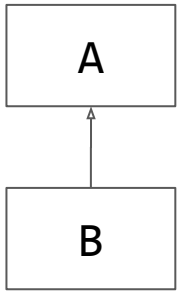
Static vs Dynamic Dispatch - Q1

```
class A:
  def f: Int = 7
  def g: Int = 2

class B extends A:
  override def f: Int = f * g
  override def g: Int = 5

val x = B()
println(x.f)
```

StackOverflowError



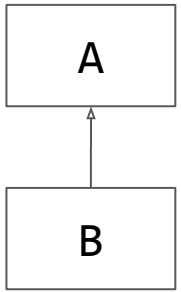
What will it print?

Static vs Dynamic Dispatch - Q2

```
class A:
  def f: Int = 7
  def g: Int = 2

class B extends A:
  override def f: Int = super.f * g
  override def g: Int = 5

val x = B()       // 35
println(x.f)
```



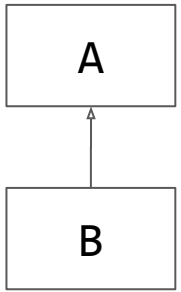
What will it print?

Static vs Dynamic Dispatch - Q3

```
class A:
  def f: Int = 7
  def g: Int = 2

class B extends A:
  override def f: Int = super.f * this.g
  override def g: Int = 5

val x: A = B()    // 35
println(x.f)
```



What will it print?

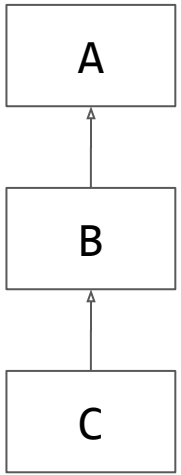
Static vs Dynamic Dispatch - Q4

```
class A:
   def f: Int = 7
   def g: Int = 2

class B extends A:
   override def f: Int = super.f * this.g
   override def g: Int = 5

class C extends B {
   override def g: Int = 6
}

val x: B = C()
println(x.f)        // 42
```



What will it print?

Static vs Dynamic Dispatch - Q5

```
class A:
   def f = 1 + this.g
   def g = 1

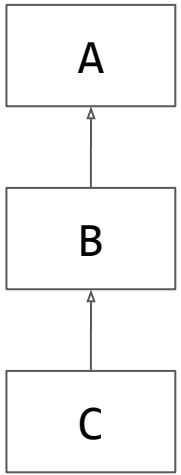
class B extends A:
   override def f = super.f * 2

class C extends B:
   override def g = 2

def test(x: A) = println(x.f)
```

```
val u: A = new C

test(u)  // 6
```



What will it print?

Static vs Dynamic Dispatch - Q6

```
class A:
   def f = 1 + this.g
   def g = 1

class B extends A:
   override def f = super.f * 2

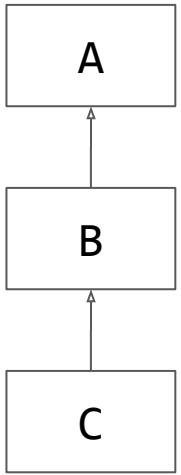
class C extends B:
   override def g = 2

def test(x: A) = println(x.f)

def test(x: C) = println(x.f + 2)
```

```
val u: A = new C

test(u)  // 6
```



What will it print?

Static vs Dynamic Dispatch - Q7

```
class A:
   def f = 1 + this.g
   def g = 1

class B extends A:
   override def f = super.f * 2

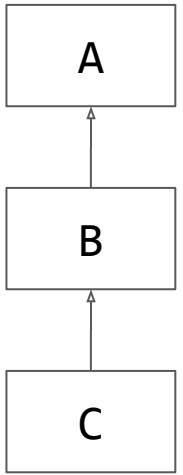
class C extends B:
   override def g = 2

def test(x: A) = println(x.f)

def test(x: C) = println(x.f + 2)
```

```
val u: B = new C

test(u)  // 6
```



What will it print?

Static vs Dynamic Dispatch - Q8

```
class A:
   def f = 1 + this.g
   def g = 1

class B extends A:
   override def f = super.f * 2

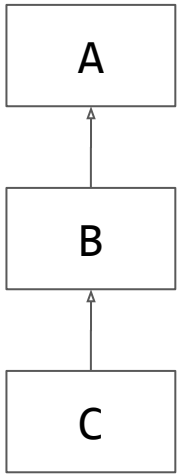
class C extends B:
   override def g = 2

def test(x: A) = println(x.f)

def test(x: C) = println(x.f + 2)
```

```
val u: C = new C

test(u)  // 8
```



What will it print?

IntList

Motivation

Create a linked list that can hold integers

```
class IntList(val head: Int, val tail: IntList):
  def including(x: Int): IntList = IntList(x, this)
```

Given the IntList, how can we create a list that contains strings?

Motivation

First try: copy & paste

```
class StringList(val head: String, val tail: StringList):
  def including(x: String): StringList = StringList(x, this)
```

- Repetitive code
- Violates DRY principle

Can we do better?

Motivation

Second try: use a super type of both `Int` and `String` (`Any` in Scala)

```
class AnyList(val head: Any, val tail: AnyList):
  def including(x: Any): AnyList = AnyList(x, this)
```

```
val xs = AnyList("one", null).including(2) // compiles without any warnings
```

```
def toUpper(xs: AnyList): AnyList =
  if xs == null then null
  else if !xs.head.isInstanceOf[String] then sys.error("Unexpected element") // extra runtime check
  else AnyList(xs.head.asInstanceOf[String].toUpperCase(), toUpper(xs.tail)) // extra cast check
```

Yes, using pattern matching the code looks better, but has the same drawbacks

- Reduced help from the compiler
- Reduced ergonomics

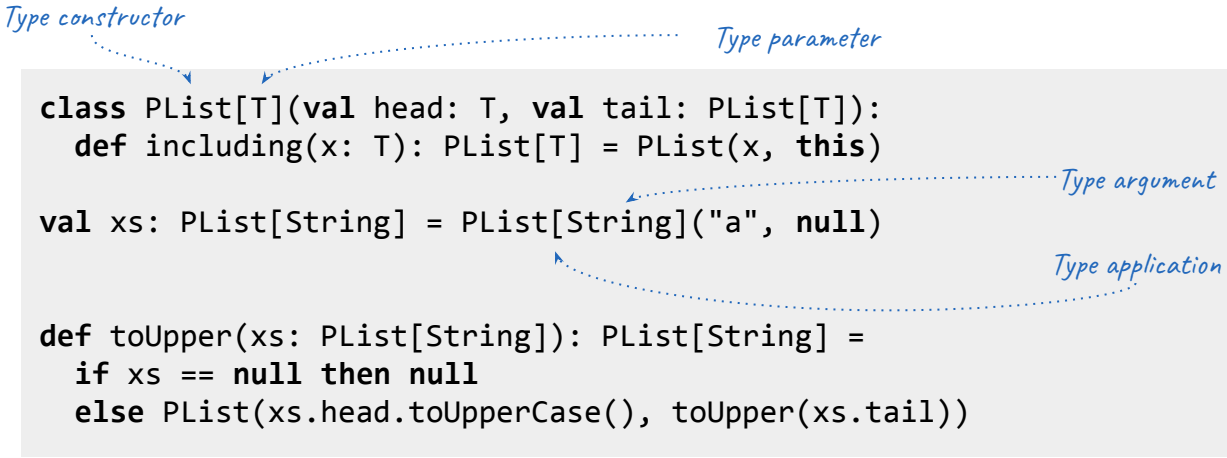
Can we do better?

Motivation

- What we want is a generic implementation that can work uniformly with any type
- Allow to make single definition work across a range of types
- Yet, maintain full static type-safety!

Motivation

- Parametric polymorphism
  - allows one to make functions and data types generic
  - maintains full static type-safety



- `PList` is a generic class
- `PList` is a type constructor (constructing new types)
- `T` is type parameter
- `PList[String]` is a type.

Parametric Polymorphism

- Classes can define one or more type parameters
  - values are provided when we define new types definitions
- Functions can define one or more type parameters
  - values are provided at call sites
- The type parameter is bound to the lexical scope that follows its definition

```
class C[T](v: T) {
  def f(x: T): T = ...
  def g[R](x: T, y: R): R = ...
}

val c: C[Int] = C[Int](10)
c.f(1)
c.g[String](1, "a")
```

```
// the type inference can often
// infer most of the type parameters

val c = C(10)
c.f(1)
c.g(1, "a")
```

How do subtypes and generics interact?

```
class A
class B extends A
```

```
class List[T]
```

`B <: A ?` 

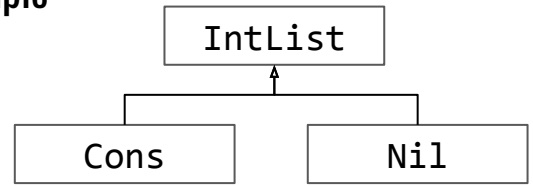
`List[B] <: List[A] ?` 

`List[B] >: List[A] ?` 

`List[B]` *has no subtype relation to* `List[A] ?` 

Polymorphism

- Two principal forms of polymorphism
  - Subtype
    - instances of subclasses can be used wherever super class is expected
    - for example



  - Parametric (Generics)
    - instances of a function or a class are created by type parameterization
    - for example `List[T]` (in Scala) or `List<T>` (in Java)

- Interaction between them
  - Bounds
  - Variance

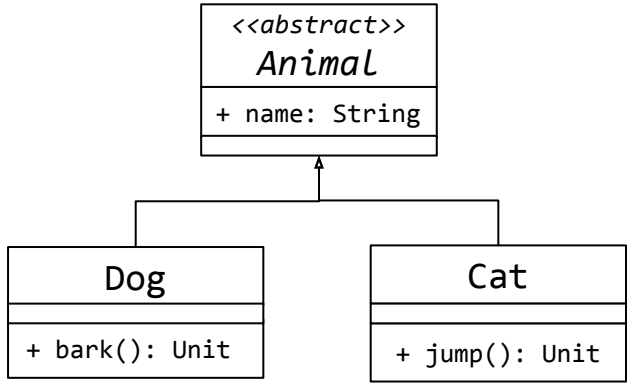
Interaction between subtyping and parametric polymorphism

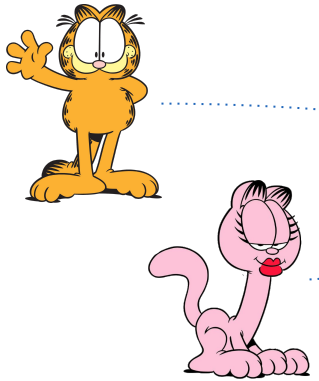
AOP - Animal Kingdom Oriented Programming

```
abstract class Animal(val name: String):
  def sleep(): Unit = println(s"$name: sleeps")

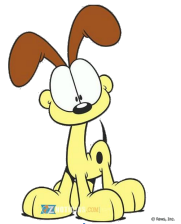
class Dog(name: String) extends Animal(name):
  def bark(): Unit = println(s"$name: barks")
  override def toString = s"Dog($name)"

class Cat(name: String) extends Animal(name):
  def jump(): Unit = println(s"$name: jumps")
  override def toString = s"Cat($name)"
```

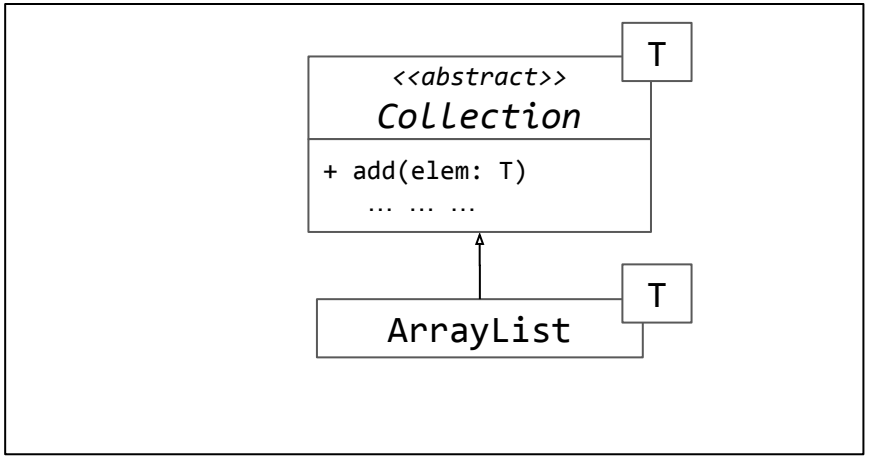




```
val garfield: Cat = Cat("Garfield")
val arlene: Cat   = Cat("Arlene")
val odie: Dog     = Dog("Odie")
```



Example generic classes



- `Collection` is an abstract mutable unordered collection
- `ArrayList` is a concrete implementation of a mutable indexed collection backed by an array

https://scastie.scala-lang.org/z4vuLpqdS5umodm2lBqg1A

Substitution Principle

```
val garfield: Cat = new Cat("Garfield")
val arlene: Cat = new Cat("Arlene")
val odie: Dog = new Dog("Odie")

val cats: ArrayList[Cat] = new ArrayList()

cats.add(garfield)
cats.add(arlene)

println(cats)
// [Cat(Garfield),Cat(Arlene)]
```

```
type mismatch;
 found: ArrayList[Cat]
 required: ArrayList[Animal]
```

```
val animal: Animal = cats.get(0)

println(animal.name)
// Cat(Garfield)

val animals: ArrayList[Animal] = cats
animals.add(odie)

println(animals)
// ???

for (cat <- cats) {
  cat.jump()
}
```

If it is, then you could write this. What would be the consequence?

Given that `Cat <: Animal` is it the case that

`ArrayList[Cat] <: ArrayList[Animal]`?

https://scastie.scala-lang.org/z4vuLpqdS5umodm2lBqg1A

Substitution Principle

```
val garfield: Cat = new Ca
val arlene: Cat = new Cat(
val odie: Dog = new Dog("C

val cats: ArrayList[Cat] =

cats.add(garfield)
cats.add(arlene)

println(cats)
// [Cat(Garfield),Cat(Arle
```

```
type m
 found
 requi
```

```
cats.get(0)
```

```
t[Animal] = cats
```

If it is, then you
this. What wou
consequen



Ar

at

]?

Bounds

Upper bound

```
class ArrayList[T] extends Collection[T]:
  def add(elem: T): Unit = ...
  // adds all src elements into the list
  def addAll(src: Collection[T]): Unit = for (e <- src) this.add(e)
```

```
val cats: ArrayList[Cat] = new ArrayList()
cats.add(garfield)
cats.add(arlene)
```

```
val animals: ArrayList[Animal] = new ArrayList()
animals.add(odie)
```

```
animals.addAll(cats)
```

```
type mismatch;
 found: ArrayList[Cat]
 required: Collection[Animal]
```

Upper bound

```
class ArrayList[T] extends Collection[T]:
  def add(elem: T): Unit = ...
  // adds all src elements into the list
  def addAll(src: Collection[_ <: T]): Unit = for (e <- src) this.add(e)
```

Upper bound

```
val cats: ArrayList[Cat] = new ArrayList()
cats.add(garfield)
cats.add(arlene)
```

`ArrayList[Cat] <: Collection[_ <: Animal]`

```
val animals: ArrayList[Animal] = new ArrayList()
animals.add(odie)

animals.addAll(cats)
```

- `[_ <: T]` means that the `src` collection may have elements of any type that is a subtype of `T`

https://scastie.scala-lang.org/vRwKCdC1R8eEQO91oARigg

Upper bound *(back to Odie)*

- Could we use the same *upper-bound trick* to make Odie jump?

```
val cats: ArrayList[Cat] = new ArrayList()

cats.add(garfield)
cats.add(arlene)

val animals: ArrayList[Animal] = cats

animals.add(odie)

for (cat <- cats) {
  cat.jump()
}
```

```
type mismatch;
 found: ArrayList[Cat]
 required: ArrayList[Animal]
```

```
val cats: ArrayList[Cat] = new ArrayList()

cats.add(garfield)
cats.add(arlene)

val animals: ArrayList[_ <: Animal] = cats

animals.add(odie)

for (cat <- cats) {
  cat.jump()
}
```

Upper bound

```
type mismatch;
 found: Dog
 required: _$1
```

https://scastie.scala-lang.org/vRwKCdC1R8eEQO91oARigg

Upper bound *(back to Odie)*

- Could we use the same *upper-bound trick* to make Odie jump?

- Cannot add a `Dog` to an `ArrayList[_ <: Animal]`
  - because we do not know the actual subtype
  - in this case it is a subtype of `Cats`

- In general, given a structure with `[_ <: T]`
  - we can get elements out (this was the case of `addAll`)
  - we cannot put elements in

```
type mismatch;
 found: ArrayList[Cat]
 required: ArrayList[Animal]
```

```
type mismatch;
 found: Dog
 required: _$1
```

Lower bound

```
class Stack[T] extends Collection[T]:
  def pop(): T = ...
  def push(elem: T): Unit = ...

  // pops all the elements into dst
  def popAll(dst: Collection[_ <: T]) =
    while (!this.isEmpty()) dst.add(this.pop())
```

It is tempting to try to use the upper bound trick

```
val cats: Stack[Cat] = new Stack()

cats.push(garfield)
cats.push(arlene)
```

```
type mismatch;
 found: ArrayList[Animal]
 required: Collection[_ <: Cat]
```

```
val animals: ArrayList[Animal] = new ArrayList()

cats.popAll(animals)
```

Lower bound

```
class Stack[T] extends Collection[T]:
  def pop(): T = ...
  def push(elem: T): Unit = ...

  // pops all the elements into dst
  def popAll(dst: Collection[_ >: T]) =
    while (!this.isEmpty()) dst.add(this.pop())
```

lower bound

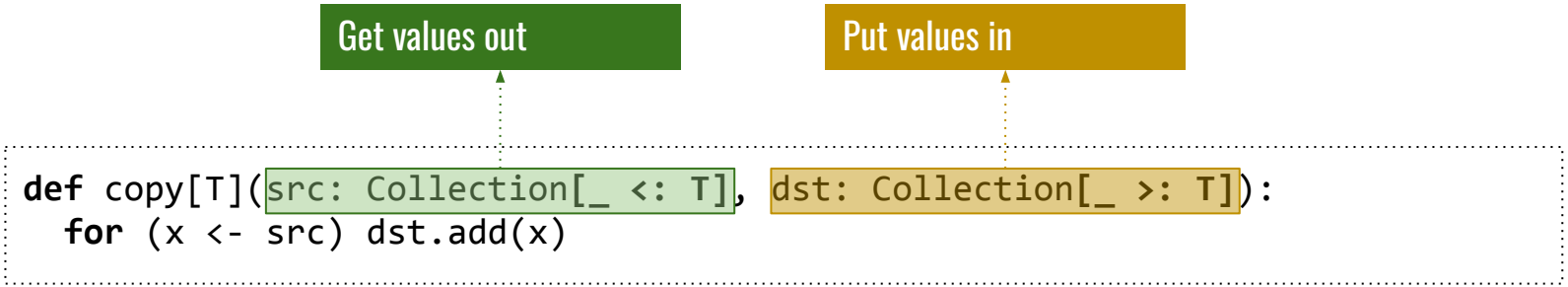
```
val cats: Stack[Cat] = new Stack()
cats.push(garfield)
cats.push(arlene)
```

ArrayList[Animal] <: Collection[_ >: Cat]

```
val animals: ArrayList[Animal] = new ArrayList()
cats.popAll(animals)
```

- `[_ >: T]` means that the `target` collection may have elements of any type that is a super type of `T`

Combining upper and lower bounds together



The Get and Put Principle

- use upper bound when you only get values out of a structure,
- use lower bound when you only put values into a structure, and
- don't use type bounds when you both get and put.

M. Naftalin and P. Wadler, Java Generics

Variance

Example

```
val cats = new ArrayList[Cat]()
cats.add(garfield)
cats.add(arlene)

val animals: ArrayList[Animal] = cats
```

```
type mismatch;
 found: ArrayList[Cat]
 required: ArrayList[Animal]
```

```
val cats: List[Cat](garfield, arlene)

val animals: List[Animal] = cats
```

List from Scala standard library

Variance

```
Cat <: Animal

List[Cat] <: List[Animal]

ArrayList[Cat] <: ArrayList[Animal]
```

Why does it work for some types and not for others?

Variance

```
class ArrayList[T]:
  def add(x: T): Unit
```

- mutable
- invariant in T

Variance annotation

```
sealed abstract class List[+A]:
```

- immutable
- covariant in T

https://scastie.scala-lang.org/tPyo0FNTTfujxwXgPNO480

Covariance *(why the mutation is the problem)*

```
class Cell[+T](init: T):
  private var current: T = init
  def get: T = current
  def set(x: T): Unit =
    current = x
```

Covariant type parameter

covariant type T occurs in contravariant position in type T of value x

```
// create a new cell with "abc"
val c1: Cell[String] = new Cell[String]("abc")
// create a new variable c2 of type Cell[Any] and assign it c (String <: Any)
val c2: Cell[Any] = c1
// set the value of the cell to 1 (Int <: Any)
c2.set(1)
// get back the value from the cell which was created a Cell[String], so string
val s: String = c1.get
```

https://scastie.scala-lang.org/tPyo0FNTTfujxwXgPNO480

Covariance *(why the mutation is the problem)*

```
class Cell[T](init: T):
  private var current: T = init
  def get: T = current
  def set(x: T): Unit =
    current = x
```

Fixing the type parameter - making it invariant

The good old error we know

```
type mismatch;
 found   : Playground.Cell[String]
 required: Playground.Cell[Any]
```

```
// create a new cell with "abc"
val c1: Cell[String] = new Cell[String]("abc")
// create a new variable c2 of type Cell[Any] and assign it c1 (String <: Any)
val c2: Cell[Any] = c1
// set the value of the cell to 1 (Int <: Any)
c2.set(1)
// get back the value from the cell which was created a Cell[String], so string
val s: String = c1.get
```

Covariance *(why the mutation isn't the only problem)*

```
class Cell[+T]:
  def test(x: T) = ()
```

```
class IntCell extends Cell[Int]:
  override def test(x: Int) = println(math.abs(x))
```

covariant type T occurs in contravariant position in type T of value x

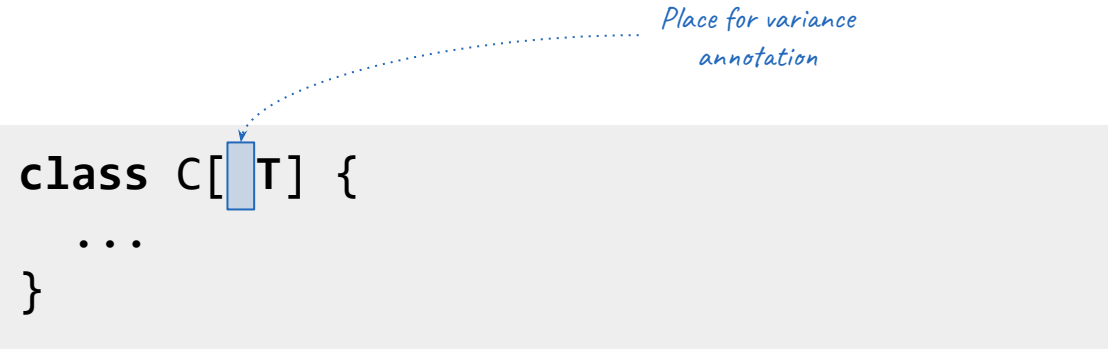
```
val xs: Cell[Int] = new IntCell
val ys: Cell[Any] = xs            // Cell[+T] ⇒ Cell[Int] <: Cell[Any]

ys.test("FIT")                    // String <: Any
```

A covariant type cannot occur in contravariant positions (method parameters, vars)

https://scastie.scala-lang.org/Kd822772RD63kE1XHNKdPg

Variance annotation



- A type parameter in a type definition can have a variance annotation
- `C[+T]` means that the class `C` is covariant in `T` *(flexible subtyping)*
  - given `A <: B` then `C[A] <: C[B]`
- `C[-T]` means that the class `C` is contravariant in `T`
  - given `A <: B` then `C[B] <: C[A]`
- `C[T]` means that the class `C` is invariant in `T` *(the default)*
  - given `A <: B` there is no relation between `C[A] and C[B]`

Bounds are not variance

- A bound (`T <: Animal`) restricts admissible type arguments.
- It also tells the compiler which operations on `T` are available.
- Even with the bound, invariant `Box[Cat]` is not a subtype of `Box[Animal]`.

Bounds vs variance: scope

- A declared bound constrains `T` at every instantiation of its definition.
- Declaration-site variance (`+T` or `-T`) determines subtyping at every use.
- A wildcard/projection (`Collection[_ <: Animal]`) restricts only that particular view.

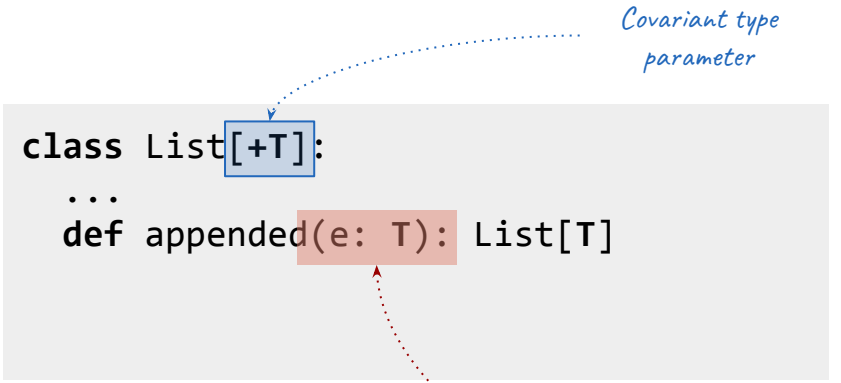
Covariance

```
val cats: List[Cat] = Nil.appended(garfield)   // Scala List

val animals = cats.appended(odie)
```

- But then how come this works?
  - How is the appended method in `List` defined that it can take both a `Cat` and a `Dog`?
  - What is the type of the result?

Covariance - solving the variance position problem

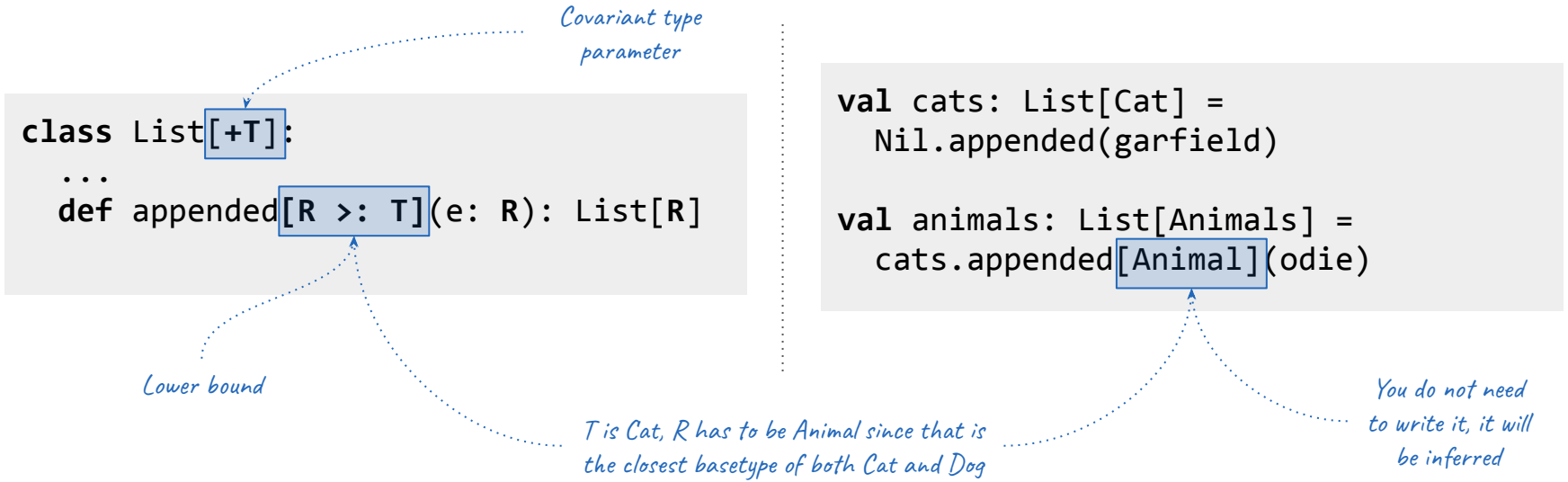



```
val cats: List[Cat] =
  Nil.appended(garfield)

val animals = cats.appended(odie)
```

```
covariant type T occurs in
contravariant position in
type T of value x
```

Covariance - solving the variance position problem



https://scastie.scala-lang.org/iQmzOT8GR3y8nWtOCkYHvg

Contravariance

```scala
class DancingCat(name: String) extends Cat(name):
  def dance(): Unit = println(s"$name: dances")
```

```scala
abstract class Playground[T]:
  def play(x: T): Unit

def sendGarfieldToPlay(out: Playground[Cat]): Unit =
  out.play(garfield)
```

```scala
class AnimalPlayground extends
  Playground[Animal]:

  override def play(x: Animal) = x.sleep()

class CatPlayground extends
  Playground[Cat]:

  override def play(x: Cat) = x.jump()

class DancingCatPlayground extends
  Playground[DancingCat]:

  override def play(x: T) = x.dance()
```

- Which of these will compile? Why?

```scala
sendGarfieldToPlay(new CatPlayground())
```

Playground[Cat] <: Playground[Cat]

```scala
sendGarfieldToPlay(new DancingCatPlayground())
```

Playground[DancingCat] <: Playground[Cat]

```scala
sendGarfieldToPlay(new AnimalPlayground())
```

Playground[Animal] <: Playground[Cat]

https://scastie.scala-lang.org/iQmzOT8GR3y8nWtOCkYHvg

Contravariance

```scala
class DancingCat(name: String) extends Cat(name):
  def dance(): Unit = println(s"$name: dances")
```

```scala
abstract class Playgroud[-T]:
  def play(x: T): Unit

def sendGarfieldToPlay(out: Playgroud[Cat]): Unit =
  out.play(garfield)
```

Contravariant type parameter

```scala
class AnimalPlayground extends
  Playground[Animal]:

  override def play(x: Animal) = x.sleep()

class CatPlayground extends
  Playground[Cat]:

  override def play(x: Cat) = x.jump()

class DancingCatPlayground extends
  Playground[DancingCat]:

  override def play(x: T) = x.dance()
```

- Which of these should compile? Why?

```scala
sendGarfieldToPlay(new CatPlayground())
```

Playground[Cat] <: Playground[Cat]

```scala
sendGarfieldToPlay(new DancingCatPlayground())
```

Playground[DancingCat] <: Playground[Cat]

```scala
sendGarfieldToPlay(new AnimalPlayground())
```

Playground[Animal] <: Playground[Cat]

Combining covariance and contravariance

A Scala function `S => T` is essentially an instance of `Function1[S, T]`

For example

```
def dblLength(x: String): Int = x.length * 2
```

Is the same as

```
val dblLength = new Function1[String, Int] {
  override def apply(x: String): Int = x.length * 2
}
```

What should be the variance annotations for `Function1` ?

```
trait Function1[?S, ?T] {
  def apply(x: S): T
}
```

Example: a web handler

- Let's try to derive that based on an example
- A router expects a function `GetRequest => Response`
- What may the router pass to the handler and read back?

```
class Request(val path: String)
class GetRequest(path: String) extends Request(path)

class Response(val status: Int)
class JsonResponse(status: Int) extends Response(status)

def route(h: GetRequest => Response): Int =
  h(GetRequest("/status")).status
```

- `GetRequest <: Request`
- `JsonResponse <: Response`
- The router supplies a GET request and reads a response.

Contravariance: accept more inputs

```
def route(h: GetRequest => Response): Int =
  h(GetRequest("/status")).status
```

- Can `Request => Response` be used for `GetRequest => Response`?

```
def generalHandler(r: Request): Response =
  Response(200)

val h: GetRequest => Response = generalHandler

route(h)
```

- Yes, `GetRequest <: Request`, so every GET request is a Request.
- A handler for all requests can also handle GET requests.
- Input contravariance reverses the subtype relation.

Covariance: return a subtype

```
def route(h: GetRequest => Response): Int =
  h(GetRequest("/status")).status
```

- Can `GetRequest => JsonResponse` be used for `GetRequest => Response`?

```
def jsonHandler(r: GetRequest): JsonResponse =
  JsonResponse(200)

val h: GetRequest => Response = jsonHandler

route(h)
```

- Yes, `JsonResponse <: Response`, so the router can use the result.
- A JSON response is also a valid general response.
- Return-type covariance preserves the subtype relation.

Combining input and output variance

```
def route(h: GetRequest => Response): Int =
  h(GetRequest("/status")).status
```

- Can `Request => JsonResponse` be used for `GetRequest => Response`?

```
def flexibleHandler(r: Request): JsonResponse =
  JsonResponse(200)

val h: GetRequest => Response = flexibleHandler
route(h)
```

- Yes! It accepts every `GetRequest` and returns a subtype of `Response`.
- A handler can accept more and return a more specific result.
- Both substitutions can happen simultaneously.

Function subtyping: test the rule

- Both input and return-type constraints must hold.

```
trait Function1[-S, +T] {
  def apply(x: S): T
}
```

- Which assignments compile? Why do the others fail?

```
val x: GetRequest => Response =
  (r: Request) => JsonResponse(200)

val y: Request => Response =
  (r: GetRequest) => Response(200)

val z: GetRequest => JsonResponse =
  (r: Request) => Response(200)
```



- `(A => B) <: (S => T)` iff `S <: A` and `B <: T`.
- Inputs are contravariant, outputs are covariant.

Combining covariance and contravariance

A Scala function `S => T` is essentially an instance of `Function1[S, T]`

```
trait Function1[-S, +T] {
  def apply(x: S): T
}
```

Covariant type produces

Contravariant type consumes

Type Erasure

Type erasure

- Type parameters and arguments do not affect evaluation
  - for compile-time only, no type parameters are present at runtime
  - the process of removing type parameters is called type erasure
  - Used in other languages (Java, Haskell, ML)
- Other strategies to deal with generics
  - Type reification
    - Type information is maintained
    - The runtime uses the generic type definition with the type arguments to create a new type (reification)
    - There is a new runtime type for each used combination of the type arguments
    - For example C#
  - Type expansion
    - The compiler uses the template definition with the template arguments and creates a new type at compile time (expands the template)
    - For example C++

Arrays in Java

- All generic types in Java are invariant and implemented using type erasure
- Except for arrays that are covariant and implemented via type reification




```
Cat[] cats = new Cat[]{ new Cat("Garfield"), new Cat("Arlene") };
Animal[] animals = cats;
animals[0] = new Dog("Odie");
Cat c = cats[0];
```

throws at runtime
ArrayStoreException(...)

- Why are arrays covariant and implemented via type reification?

```
class Arrays {
  void sort(Object[] arr) {...}
  String toString(Object[] arr) {...}
  ...
}
```

Generics across languages

Generics: three separate questions

- What arguments are valid? Bounds, constraints, concepts.
- When are different instantiations substitutable? Variance and projections.
- How is code implemented? Erasure, runtime reification, or specialization.

One example across languages

- Derived inherits from Base. We create a collection of Derived objects.
- Can we use it where a collection of Base is expected for reading?
- Can we also add an arbitrary Base? Which conversions are safe?

Java: invariant, with use-site wildcards

- Java's List<T> is invariant, independently of the element hierarchy.

```
import java.util.*;
class Base {}
class Derived extends Base {}
```

- Why can we read Base values but not insert them?

```
List<Derived> ds = new ArrayList<>();
List<Base> bs = ds;                    // ERROR
List<? extends Base> view = ds;        // OK
Base item = view.get(0);               // OK
view.add(new Base());                  // ERROR
```

- The wildcard changes this view, not List<T> itself.
- Reading is safe; adding a Base may corrupt a List<Derived>.

Java: bounded parameters vs wildcards

- Two uses of 'extends' answer different questions.

```
class Box<T extends Base> {
  T value;
}
```

- Which constraint applies to all Boxes, and which only here?

```
Box<Derived> d = new Box<>();
Box<Base> b = d;                    // ERROR
Box<? extends Base> read = d;       // OK

List<? super Derived> dst = new ArrayList<Base>();
dst.add(new Derived());             // OK
```

- T extends Base restricts every Box<T> instantiation.
- ? extends / ? super restrict individual uses (PECS).

C++: no container covariance

- The containers are unrelated; the algorithm is generic.

```
#include <vector>
struct Base { virtual ~Base() = default; };
struct Derived : Base {};
```

- Why can inspect(ds) work without vector covariance?

```
template<class T>
void inspect(const std::vector<T>& xs) {
  for (const Base& x : xs) { … }
}
void demo() {
  std::vector<Derived> ds;
  inspect(ds);                    // OK
  std::vector<Base> bs = ds;      // ERROR
}
```

- Template deduction instantiates inspect<Derived>.
- Each Derived is readable as Base; no vector conversion occurs.

C++20: a constraint is not variance

- Constrain the same generic algorithm to subtypes of Base.

```
#include <concepts>
// Reuse Base, Derived and std::vector
```

- Does std::derived_from make vector<T> covariant?

```
template<class T> requires std::derived_from<T, Base>
void inspect(const std::vector<T>& xs) {
  for (const Base& x : xs) { … }
}

void demo() {
  inspect(std::vector<Derived>{}); // OK
  inspect(std::vector<int>{});     // ERROR
}
```

- No. The concept checks admissible T at each instantiation.
- vector<Derived> still is not a subtype of vector<Base>.

Kotlin: covariant read-only lists

- Kotlin specifies out T on the List declaration.

```
open class Base
class Derived : Base()
```

- Why is List covariant but MutableList invariant?

```
val ds: List<Derived> = listOf(Derived())
val bs: List<Base> = ds                    // OK

val ms: MutableList<Derived> = mutableListOf(Derived())
val mb: MutableList<Base> = ms             // ERROR
val view: MutableList<out Base> = ms       // OK
view.add(Base())                           // ERROR
```

- List<out T> is a read-only, covariant interface.
- MutableList<T> is invariant; out projection restricts a view.

C#: variant interfaces, invariant lists

- C# allows variance on interfaces/delegates, not List<T>.

```
class Base {}
class Derived : Base {}
```

- Why can IEnumerable<Base> read ds but List<Base> cannot?

```
List<Derived> ds = new();
List<Base> bs = ds;                    // ERROR
IEnumerable<Base> read = ds;           // OK

Action<Base> acceptBase = _ => {};
Action<Derived> accept = acceptBase;   // OK
```

- `IEnumerable<out T>` is covariant
- `List<T>` is invariant.
- `Action<in T>` is contravariant; no use-site wildcards.

Rust: variance is mostly about lifetimes

- Rust has no general class-subtype hierarchy like Scala/Java/C++/...
- It has subtyping in terms of lifetimes
- `&'static str <: &'a str`
  - a reference that lives forever can be used anywhere a shorter is expected
- For a type constructor `C<T>` the question is whether `W<&'static str> <: W<&'a str>`
  - which depends on variance of `W`
- Rust compiler runs variance inference on every generic type definition look at where each type parameter appears in the fields
  - &'a T: covariant in 'a and T.
  - &'a mut T: covariant in 'a, invariant in T.
  - and other rules...

Summary

Summary: Generics

- Type parameters let classes and methods work uniformly across types.
- Type arguments may be written explicitly or inferred by the compiler.
- Subtyping does not automatically lift: `C[Cat]` need not be a subtype of `C[Animal]`.

Summary: Bounds

- Upper bounds (`<: T`): read values safely from a producer.
- Lower bounds (`>: T`): write values safely to a consumer.
- Get/put principle: use producers for reading, consumers for writing.

Summary: Variance

- `C[+T]` is covariant: subtyping is preserved (safe output).
- `C[-T]` is contravariant: subtyping is reversed (safe input).
- `C[T]` is invariant
- `Function1[-S, +T]` combines both directions.

Summary: Implementation

- Type erasure removes generic type arguments at runtime (Java, Scala).
- Reification retains type information (C# generics; Java arrays).
- Expansion specializes code at compile time (C++ templates).

Questions

Take-away: Types

- Why is mutable `ArrayList[Cat]` not a subtype of `ArrayList[Animal]`?
- When can `C[Cat]` be substituted for `C[Animal]`?

Take-away: Bounds

- What may you read or insert through `Collection[_ <: Animal]`?
- Which bounds belong on the source and destination of `copy[T]`?
- Why can `List[+T].appended[R >: T](x: R)` accept a `Dog` after `Cat`?

Take-away: Functions and Runtime

- Is `(Animal => Cat)` a subtype of `(Cat => Animal)`? Explain.
- Why can Java covariant arrays throw ArrayStoreException?
- How do erasure, reification, and expansion differ?